

Report – IN3030 Oblig 5

Processor:

- Apple M1-chip, 3228 MHz
- 8 core cpu

Introduction

In this report we will make a program that uses threads to count with a little twist. Every odd number has to wait for the next odd number to “release it” whilst every even number can just print out. The Program starts from 1, so the output should look like 2,1,4,3,6,5,8,7 and so on.

I interpreted that we were supposed to make a new thread for each number since in the task assignment it referred to each iteration as its own thread.

User Guide

1. To first start the program, you have to compile the java file using:
javac WaitAndSwap.java
2. After, you want to run the program using:
java WaitAndSwap <num iterations>

“Num iterations” describes how many iterations will be done by the program.

Synchronization

To sync the program I used semaphores like in the “WaitNextC.java”. I ended up using three different semaphores for the task, but used an additional to help out the output:

Mutex Semaphore (mutex): Ensures that incrementing the shared count variable is an atomic operation.

Release Semaphore (releaseSemaphore): Manages the release order of threads.

Wait Semaphore (waitSemaphore): Controls which threads need to wait based on the sequence of calls.

Write Semaphore (writeSemaphore): Ensures that threads writing to files do not collide.

Implementation

The program doesn’t look that different from the “WaitNextC.java” program we were given to use as inspiration since our program uses both “variSpeed” and “debugPrintln”. Each thread is identified by a unique index, thereafter calls waitAndSwap upon execution. Depending on its order of arrival (determined by the count), it either waits or proceeds immediately and potentially releases a previously waiting thread.

By using WaitNextC.java methods for debugging we can see in the terminal the output.

Conclusion

The WaitAndSwap program implements a synchronization mechanism using only semaphores. It demonstrates effective thread management and communication, ensuring that the threads execute in a predetermined order without interference.